FORD-FULKERSON ALGORITHM ● SIDE 1: CODE

```
#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

int capacity_[MAX][MAX]; // residual capacity graph
int parent[MAX];

void inputEdgesToAdjMatrix(int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            capacity_[i][j] = 0;
    cout << "Enter edges (Source Destination Capacity):\n";
    for (int i = 0; i < e; i++) {
        int u, v, c;
        cin >> u >> v >> c;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v && c > 0) {
            capacity_[u - 1][v - 1] += c; // += for parallel edges
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}
// Manual BFS to find an augmenting path in the residual graph
bool bfsFindPath(int n, int source, int sink) {
    bool visited[MAX] = {false};
    int queueArr[MAX];
    int qFront = 0, qRear = 0;
    queueArr[qRear++] = source;
    visited[source] = true;
    parent[source] = -1;

    while (qFront < qRear) {
        int u = queueArr[qFront++];
        for (int v = 0; v < n; v++) {
            // residual edge exists if capacity remaining > 0
            if (!visited[v] && capacity_[u][v] > 0) {
                queueArr[qRear++] = v;
                visited[v] = true;
                parent[v] = u;
                if (v == sink) return true;
            }
        }
    }
    return false;
}
```

```
// Ford-Fulkerson (implemented via BFS = Edmonds-Karp variant)
void fordFulkerson(int n, int source, int sink) {
    int maxFlow = 0, pathNumber = 1;

    while (bfsFindPath(n, source, sink)) {
        // find bottleneck capacity along this augmenting path
        int pathFlow = INF;
        int v = sink;
        while (v != source) {
            int u = parent[v];
            if (capacity_[u][v] < pathFlow) pathFlow = capacity_[u][v];
            v = u;
        }
        cout << "\nAugmenting Path " << pathNumber << ": ";
        int pathNodes[MAX], len = 0;
        v = sink;
        while (v != -1) { pathNodes[len++] = v; v = parent[v]; }
        for (int i = len - 1; i >= 0; i--) {
            cout << pathNodes[i] + 1;
            if (i != 0) cout << " -> ";
        }
        cout << "  (Bottleneck = " << pathFlow << ")\n";

        // update residual capacities along the path
        v = sink;
        while (v != source) {
            int u = parent[v];
            capacity_[u][v] -= pathFlow; // forward: reduce remaining
            capacity_[v][u] += pathFlow; // backward: allow undo
            v = u;
        }
        maxFlow += pathFlow;
        pathNumber++;
    }
    cout << "\nMaximum Flow = " << maxFlow << endl;
}
int main() {
    int n, e;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(n, e);
    int source, sink;
    cout << "\nEnter Source node: ";
    cin >> source;
    cout << "Enter Sink node: ";
    cin >> sink;
    fordFulkerson(n, source - 1, sink - 1);
    return 0;
}
```

FORD-FULKERSON ALGORITHM ● SIDE 2: WHEN, HOW & GOTCHAS

WHEN TO USE FORD-FULKERSON (Scenario Triggers)

- Question asks for MAXIMUM FLOW through a network (pipes, roads, data, traffic)
- Question mentions Source (S), Sink (T), and edge CAPACITIES (not weights/costs)
- "Maximum amount of water/data/traffic that can flow from A to B" style scenarios
- Question asks for MINIMUM CUT — the smallest total capacity that, if removed, disconnects source from sink
- Network has multiple possible paths from source to sink, each with a capacity limit

Key Vocabulary (from Class Notes)

- Source (S) / Sink (T) — where flow starts / where flow must end up
- Capacity — maximum flow an edge can carry. Flow — actual amount currently using that edge
- Augmenting Path — any S-to-T path in the RESIDUAL graph with spare capacity remaining
- Bottleneck Capacity — the SMALLEST remaining capacity along an augmenting path (limits how much flow that path can add)
- Residual Graph — shows REMAINING usable capacity on each edge, INCLUDING reverse/backward edges
- Max-Flow = Min-Cut — this is ALWAYS true (a core theorem); max flow value equals the minimum cut capacity

MANUAL TRACE METHOD (Augmenting Path by Path, from Class Notes)

Use this exact process — matches the class notes' path-listing + bottleneck style:

- 1. Find ANY path from S to T where every edge still has spare capacity (residual > 0).
- 2. The BOTTLENECK of this path = the smallest remaining capacity among its edges.
- 3. Add this bottleneck value to the total flow. Write it as: S→...→T = [bottleneck value].
- 4. Update the residual graph: REDUCE forward capacity by bottleneck, INCREASE backward (reverse) capacity by bottleneck.
- 5. Repeat steps 1-4, finding a NEW augmenting path each time, until NO path from S to T has any remaining capacity.
- 6. Sum ALL bottleneck values used = Total Maximum Flow.
- 7. For MIN CUT: after no augmenting path remains, find all nodes STILL reachable from S in the residual graph — call this set 'S-side'. All other nodes = 'T-side'. Min cut = sum of original capacities of edges crossing from S-side to T-side.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Backward/reverse edges are ESSENTIAL — they let the algorithm 'undo' a bad earlier flow choice by rerouting. Forgetting to add reverse capacity breaks correctness.

Empty vs non-empty backward edge (from class notes diagram) — a backward edge only allows flow if there's actual forward flow to 'cancel'; capacity_[v][u] starts at 0 and grows as forward flow is pushed.

Full vs non-full forward edge — once capacity_[u][v] reaches 0, that edge can no longer be used in this direction until some flow is undone via the reverse edge.

Order of paths found does NOT affect the final max-flow VALUE, but does affect which specific paths/flows are shown as the 'answer' — both are valid.

Min-Cut nodes are found using BFS/DFS reachability on the FINAL residual graph — this is explicitly noted in class notes as '[BFS]' next to Min Cut.

FORD-FULKERSON ALGORITHM ● SIDE 3: WORKED EXAMPLES

EXAMPLE 1 — Small Network (Trace by Hand, verified against code)

Graph (directed, capacities): 1->2=16, 1->3=13, 2->3=10, 3->2=4, 2->4=12, 4->3=9, 3->5=14, 5->4=7, 4->6=20, 5->6=4

Source=1, Sink=6 (classic CLRS max-flow example)

Step-by-step trace:

- Augmenting Path 1: 1 -> 2 -> 4 -> 6 (Bottleneck = min(16,12,20) = 12)
- Augmenting Path 2: 1 -> 3 -> 5 -> 6 (Bottleneck = min(13,14,4) = 4)
- Augmenting Path 3: 1 -> 3 -> 5 -> 4 -> 6 (Bottleneck = min(9,10,7,8) = 7)
- No more augmenting paths remain in the residual graph → STOP

Maximum Flow = 12 + 4 + 7 = 23 (verified against compiled code output)

EXAMPLE 2 — Finding the Min Cut

Using the SAME network as Example 1, after max flow = 23 is found:

Check which nodes are still reachable from Source(1) in the final residual graph — this defines the S-side of the cut. All remaining nodes form the T-side.

Exam expects you to say: "Sum the ORIGINAL capacities of all edges crossing from S-side to T-side. This sum will equal the max flow (23), confirming the Max-Flow = Min-Cut theorem."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A water utility company has a pipe network from a reservoir to a city, with each pipe having a maximum flow capacity (liters/second). What is the maximum water flow achievable to the city?"

How to apply Ford-Fulkerson: reservoir = source, city = sink, pipe junctions = nodes, pipe capacities = edge capacities. Repeatedly find augmenting paths with spare capacity, push the bottleneck amount of flow, until no more augmenting paths exist. The final total = maximum deliverable water flow.

EXAMPLE 4 — Bottleneck Identification Twist (Medium/Hard style)

Scenario: exam gives a specific augmenting path and asks "what is the bottleneck, and which edge causes it?"

Correct handling: identify the SMALLEST remaining (residual) capacity among all edges on that specific path — not the smallest ORIGINAL capacity if some edges already have reduced flow from a previous augmenting path. Always check the CURRENT residual capacity, not the original graph's capacity, since earlier paths may have already used up part of an edge's capacity.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Ford-Fulkerson answer, check:

Did I use CURRENT residual capacities (not original) when finding each new bottleneck?

Did I correctly update BOTH forward (decrease) and backward (increase) capacities after each path?

Did I keep finding paths until truly NONE remain (not stop early)?

If asked for Min Cut, did I use BFS/DFS reachability on the FINAL residual graph?

Does my Min Cut capacity sum EQUAL my Max Flow value? (They must always match.)